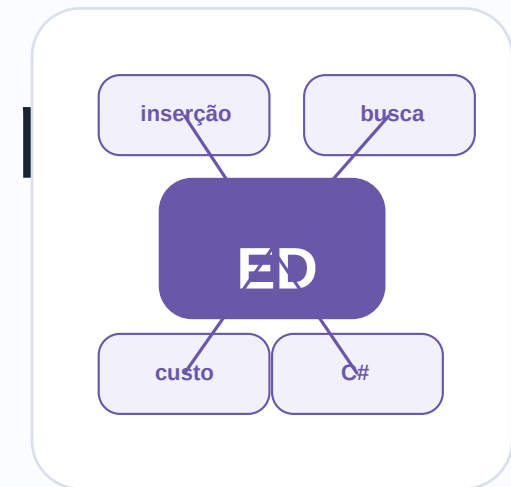


Lista encadeada: requisitos de implementação

Como programar inserção no início, no meio e no fim, além de busca em diferentes posições.



inserção

busca

custo

C#

Estrutura base

A lista guarda início, fim e tamanho para controlar a sequência de nós.

Operações

Inserir no início, inserir no meio, inserir no fim e buscar valores.

Regra geral

Como cada nó aponta só para o próximo, várias operações caminham desde o início.

No, início, fim e tamanho

Antes das operações, a lista precisa de uma classe No e referências de controle.

Estrutura inicial

```
public class No
{
    public int Valor;
    public No Prox;

    public No(int valor)
    {
        Valor = valor;
        Prox = null;
    }
}

No inicio = null;
No fim = null;
int tamanho = 0;
```

início

Aponta para o primeiro nó da lista.

fim e tamanho

fim acelera inserção final. tamanho ajuda a validar posições.

Novo nó vira o primeiro

É o caso mais simples: não precisa percorrer a lista.

InserirInicio

```
public void InserirInicio(int valor)
{
    No novo = new No(valor);
    novo.Prox = inicio;
    inicio = novo;

    if (fim == null)
        fim = novo;

    tamanho++;
}
```



Passos

Criar o nó, ligar novo.Prox ao antigo início e atualizar início.

Custo

$O(1)$, porque não depende da quantidade de nós.

Precisa parar no nó anterior

Para inserir em uma posição intermediária, encontre primeiro quem vem antes dela.

InserirNaPosicao

```
public void InserirNaPosicao(int valor, int pos)
{
    if (pos < 0 || pos > tamanho)
        throw new ArgumentOutOfRangeException();

    if (pos == 0) { InserirInicio(valor); return; }
    if (pos == tamanho) { InserirFim(valor); return; }

    No anterior = inicio;
    for (int i = 0; i < pos - 1; i++)
        anterior = anterior.Prox;

    No novo = new No(valor);
    novo.Prox = anterior.Prox;
    anterior.Prox = novo;
    tamanho++;
}
```

Passo-chave

Pare no nó anterior. Ele é quem receberá a ligação para o novo nó.

Custo

$O(n)$, porque pode ser necessário caminhar por parte da lista.

Novo nó vira o último

Com ponteiro fim, a operação fica direta. Sem ele, seria preciso percorrer até o último nó.

InserirFim

```
public void InserirFim(int valor)
{
    No novo = new No(valor);

    if (inicio == null)
    {
        inicio = novo;
        fim = novo;
    }
    else
    {
        fim.Prox = novo;
        fim = novo;
    }

    tamanho++;
}
```

Lista vazia

inicio e fim apontam para o novo nó.

Lista com nós

fim.Prox aponta para novo, depois fim passa a ser novo.

Uma função resolve início, meio e fim

A busca compara o valor do nó atual. Se não achou, avança para o próximo.

Buscar

```
public No Buscar(int valor)
{
    No atual = inicio;

    while (atual != null)
    {
        if (atual.Valor == valor)
            return atual;

        atual = atual.Prox;
    }

    return null;
}
```

Ideia

A mesma função encontra valores no início, no meio ou no fim.

O que muda

Muda a quantidade de passos até encontrar o valor ou chegar em null.

Melhor caso

Quando o valor está no primeiro nó, a busca compara uma única vez e termina.



Como funciona

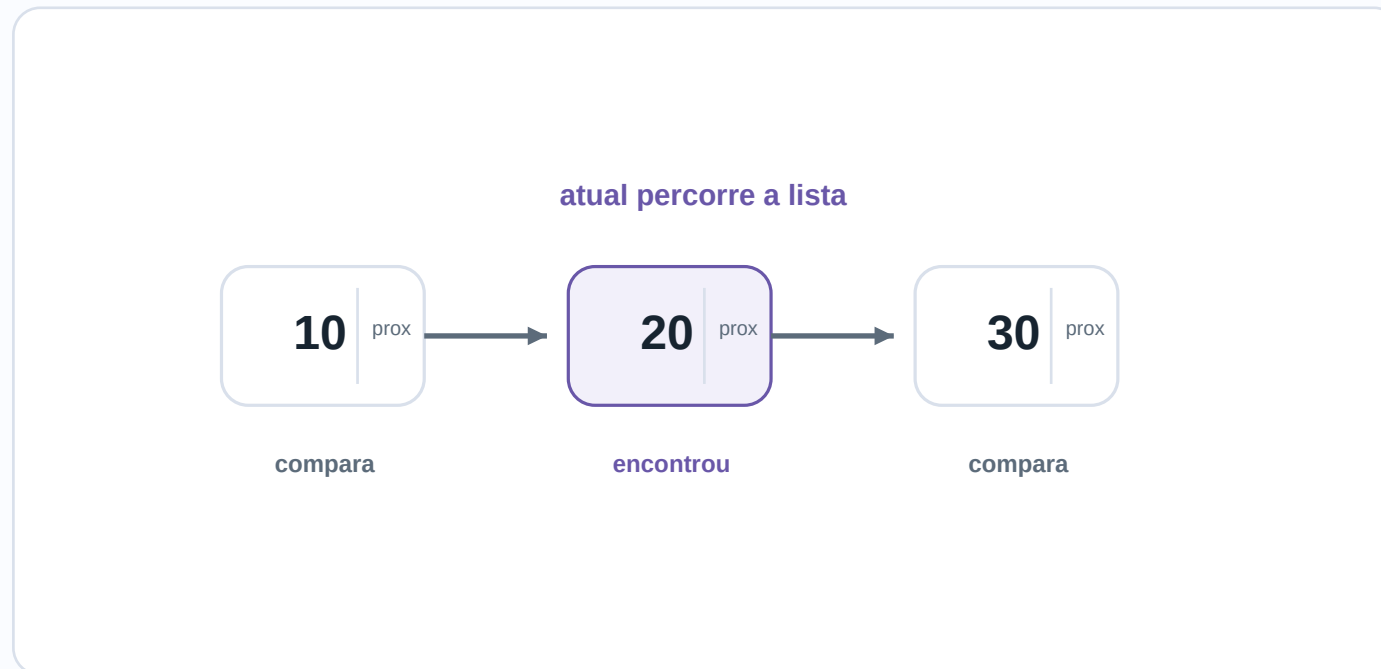
atual começa em início. atual.Valor já é o valor procurado.

Custo

$O(1)$, porque não precisa andar pela lista.

Percurso parcial

Quando o valor está no meio, a busca passa pelos nós anteriores antes de encontrar.



Como funciona

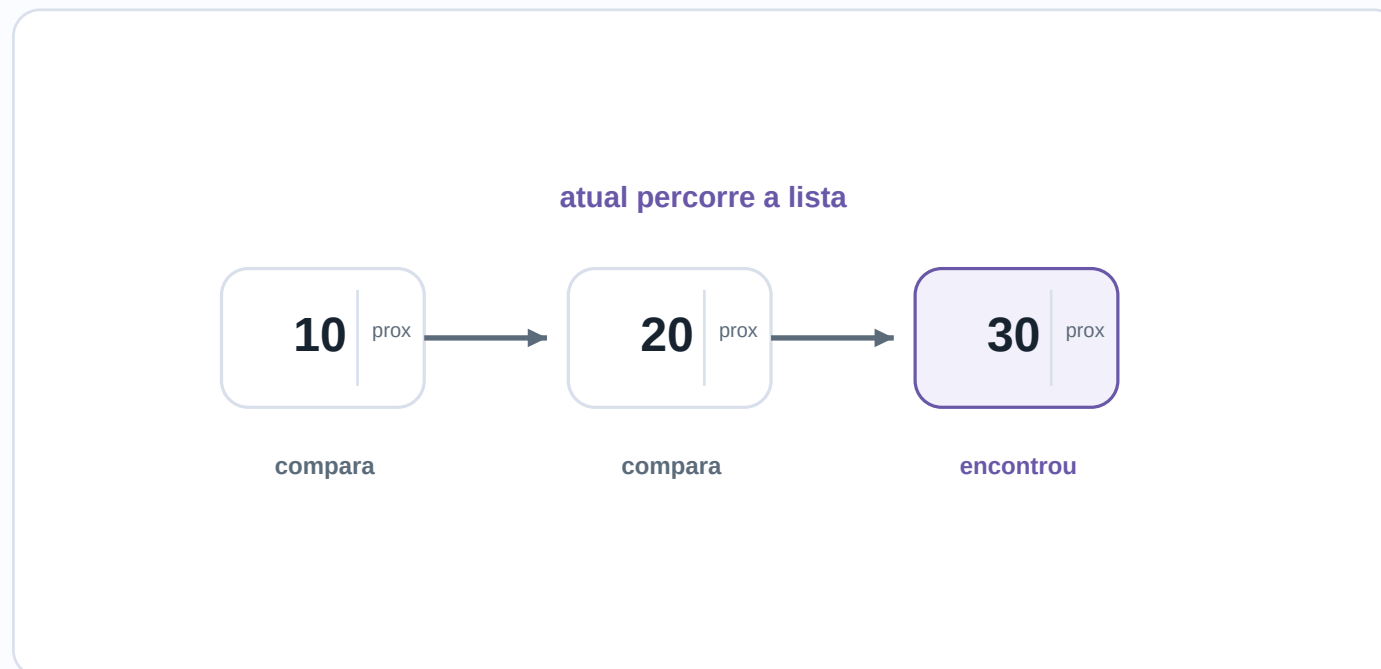
Compara com o primeiro nó, avança, e repete até chegar no valor.

Custo

$O(k)$, onde k é a posição visitada; em análise geral, tratamos como $O(n)$.

Pior caso com valor existente

Quando o valor está no último nó, a busca atravessa quase toda a lista.



Como funciona

Compara desde o início e avança nó por nó até o último elemento.

Custo

$O(n)$, porque pode precisar visitar todos os nós.

O que muda entre início, meio e fim

O segredo é saber quando precisa percorrer a lista e quando basta mexer nas referências diretas.

Início

Inserção é rápida. Busca é melhor caso quando o valor já está no primeiro nó.

Meio

Normalmente precisa percorrer até o ponto certo antes de inserir ou encontrar.

Fim

Inserção fica rápida com ponteiro fim. Busca no fim continua exigindo percurso.

Prova no código

Use exemplos no Main chamando inserir e buscar para início, meio, fim e valor inexistente.